COLMAP SfM and Gaussian Splatting on South Building

Sindhu Kaleeswaran | 3703126779

1. Introduction and setup

This report evaluates a two-stage novel-view pipeline using the South Building dataset. COLMAP structure-from-motion (SfM) recovers camera poses and a sparse point cloud and 3D Gaussian Splatting (GS) is then trained on those poses. Experiments use the provided 'images15' and 'images30' subsets (1600x1200).

All experiments ran on Google Colab with a Tesla T4 GPU. COLMAP was run from the command line with the SIMPLE_PINHOLE model and one shared camera. GS used the official graphdeco-inria implementation.

2. Structure from Motion with COLMAP

Each configuration ran feature extraction, matching and incremental mapping from a clean workspace. Statistics come from 'model_analyzer'. Exhaustive matching compares all N(N-1)/2 image pairs, while sequential matching compares each image only with its 10 nearest neighbours in filename order.

2.1 Quantitative results

Table 1: COLMAP results per configuration (SIMPLE_PINHOLE)

Configuration	Registered	3D points	Mean track length	Mean reproj. error (px)	Extract (s)	Match (s)	Map (s)	Total (s)
15-exhaustive	15 / 15	13,364	3.80	0.304	76.0	157.5	24.2	257.7
15-sequential	15 / 15	13,314	3.80	0.302	76.9	135.6	22.7	235.1
30-sequential	30 / 30	32,755	5.24	0.331	147.3	341.6	100.7	589.6
30-exhaustive	30 / 30	33,207	5.20	0.335	149.0	626.7	113.7	889.3

All four runs registered every image into a single model with sub-pixel reprojection error. The self-calibrated focal length was consistent across runs (1341–1345 px, 0.27% spread), well below COLMAP's 1920 px default guess for images without EXIF.

2.2 Qualitative results



Figure 1: 12,375 SIFT keypoints detected on image #1.

Features are dense on brick, window frames and trees/bushes, and nearly absent on the textureless sky.



Figure 2: Neighbouring views #1 and #2: 3,984 of 4,021 raw matches (99%) survive geometric verification.



Figure 3: Distant views #1 and #12: all 32 raw matches are rejected by RANSAC.

Matches between neighbouring views are almost all geometrically consistent. The few matches between different faces of the building are coincidental (repeated bricks and windows) and are correctly discarded.

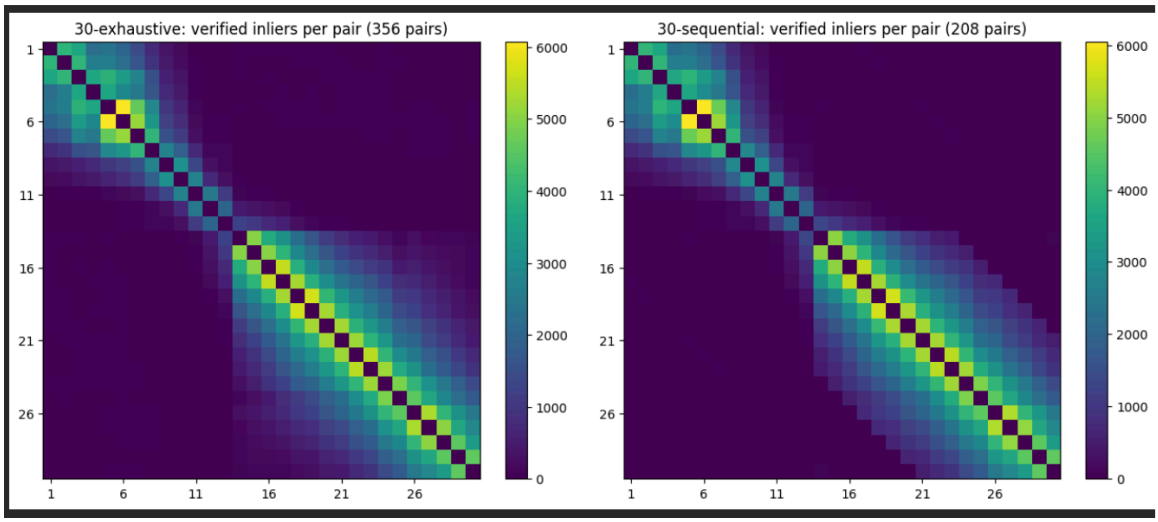


Figure 4: Verified inliers per image pair for 30 images: exhaustive (356 pairs) vs. sequential (208 pairs).

The inlier matrix forms two blocks (images 1-10 and 13-30), one per facade, joined only by a few corner views around #10-#13. This corner is the weakest link in the reconstruction. The pairs that sequential matching skips carry few inliers, which explains why it loses almost nothing.

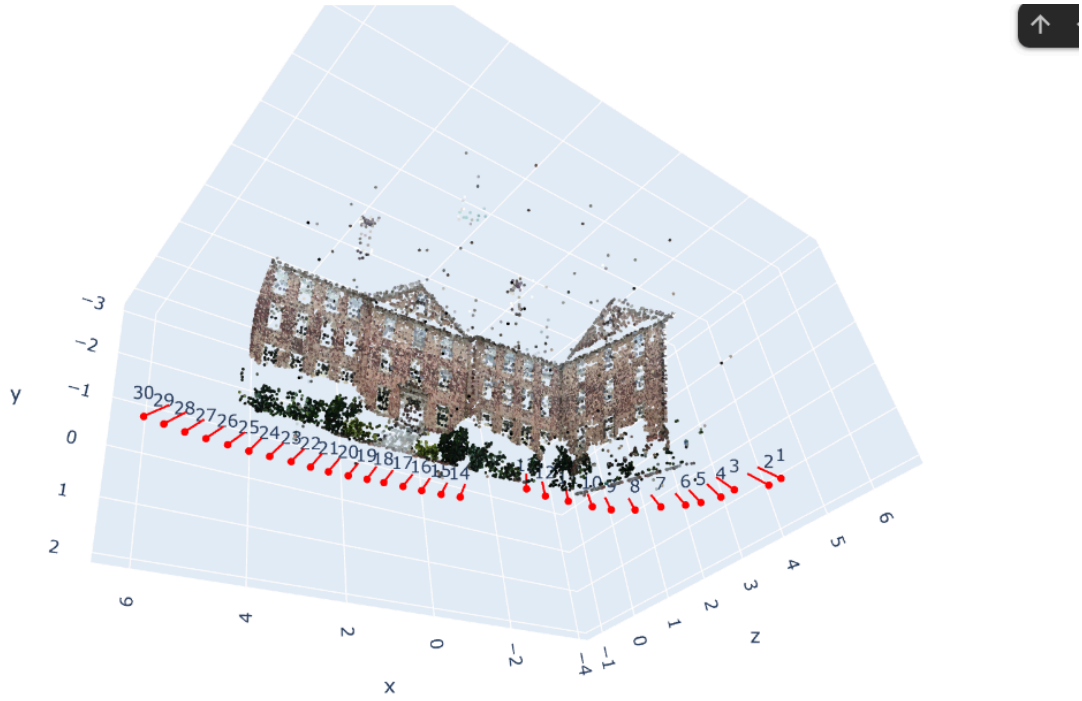


Figure 5: Sparse point cloud and camera poses (30 images, sequential).

In the 3D view, both roof-sections, the roofline and the sunken window areas (gaps in the facade) are recognizable. A few scattered points float above the building: sky, roof edge and tree outliers.

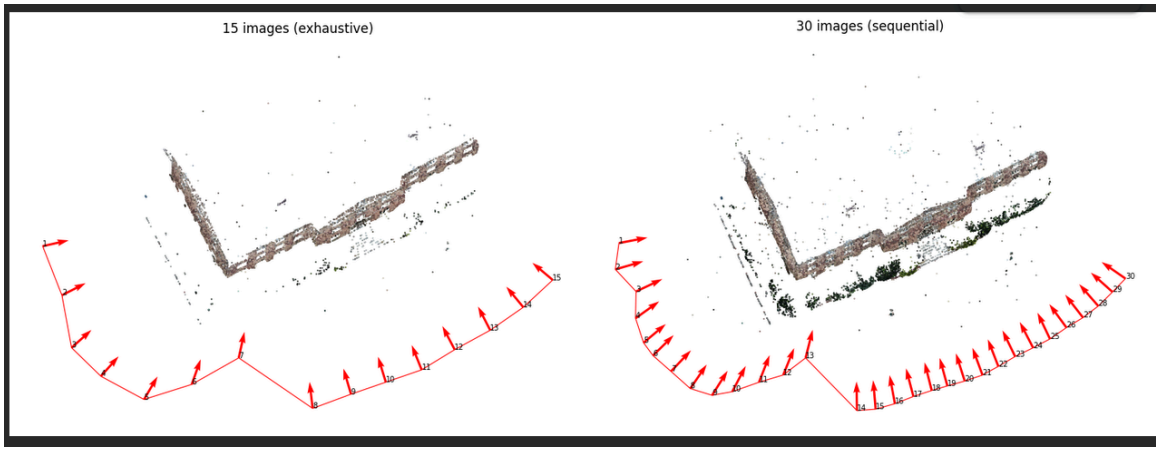


Figure 6: Top-down view of the sparse model and camera path, 15 vs 30 images.

From above, both walls are thin, straight and meet at roughly a right angle, which indicates accurate poses and intrinsics; drift would bend or thicken them. The step on the long wall appears in both independent models, so it is real geometry. The largest gap in the camera path (#13 -> #14) coincides with the weak link in Figure 4, and the 30-image model is visibly denser.

2.3 Discussion

Table 2: Sequential vs. exhaustive matching.

Measure	15 images	30 images
Matching time saved by sequential	13.9%	45.5%
3D points retained by sequential	99.6%	98.6%
Change in reprojection error	−0.001 px	−0.004 px

Matching mode. At 15 images the two modes are nearly identical, since a 10-image window already covers about 95 of the 105 pairs. At 30 images, sequential saves about 5 minutes for a 1.4% loss in points. Matching time scales with the number of pairs tried, so the advantage grows with dataset size. This relies on the images being in capture order; for unordered images, exhaustive matching would be required.

Dataset size. Doubling the images gave 2.5x more points and longer tracks (3.80 -> 5.20), because closer consecutive views let each point be observed by more cameras. Reprojection error rose slightly (0.30 -> 0.34 px), which reflects more constraints per point rather than lower quality. Mapping time grew 4.7x, since bundle adjustment scales with the number of observations.

Completeness. Every image registered into a single model, so the low reprojection error is meaningful and no alternative camera model or guided matching was needed. For Part 2 we used the 30-image exhaustive model, which has the most points and all loop pairs verified.

3. Gaussian Splatting

3.1 Setup

We trained one GS model on the 30-image exhaustive reconstruction, holding out images #10, #20 and #30 (P1180198, P1180208, P1180218). As specified, ‘llffdhold’ in ‘readColmapSceneInfo’ was set to 0 and the three filenames were listed in ‘sparse/0/test.txt’, giving 27 training and 3 test views. Training ran for 6000 iterations at full 1600x1200 resolution with default parameters, saving the model at iterations 2000 and 6000. Both splits were rendered from each saved model. Test metrics come from ‘metrics.py’; training-split metrics use the repository's own PSNR and SSIM functions, which reproduce the ‘metrics.py’ test values exactly.

3.2 Quantitative results

Table 3: Mean image quality by split and iteration (LPIPS computed for the test split only).

Split	Iteration	PSNR (dB) (inc)	SSIM (inc)	LPIPS (dec)
Train (27 views)	2000	23.88	0.702	–
Train (27 views)	6000	27.29	0.841	–
Test (3 views)	2000	19.94	0.654	0.367
Test (3 views)	6000	22.29	0.766	0.221

Table 4: Results on the held-out test images.

View	PSNR @2000	PSNR @6000	SSIM @2000	SSIM @6000
#10	23.60	22.56	0.688	0.759
#20	24.57	26.81	0.722	0.840
#30	11.66	17.49	0.552	0.699

All mean metrics improve from 2000 to 6000 iterations: test PSNR rises by 2.3 dB and LPIPS drops by 40%, reflecting much sharper renders. The train and test gap widens from 3.9 to 5.0 dB, as the model fits its 27 training views faster than it generalizes. The test mean hides a large spread: #20 matches training quality, while #30 is 9 dB lower. #10 is the only view whose PSNR decreases, even though its SSIM improves.

3.3 Qualitative results



Figure 7: Held-out views: ground truth, renders at 2000 and 6000 iterations and absolute error at 6000 (brighter = larger error).

Between 2000 and 6000 iterations, blurred bushes/trees and smeared frame borders become sharp brick, window frames and individual bushes. The quality of each novel view depends mainly on camera coverage:

- **#20 (interpolation, strongest).** A frontal mid-facade view whose neighbours #19 and #21 are both in training. The remaining errors are on window reflections, which change with viewpoint, and on bushes.
- **#10 (corner view).** It lies at the corner where Part 1 showed the weakest coverage, and errors concentrate on the bushes and edge trees. At 6000 the model renders sharp but misaligned leaf texture: PSNR penalizes this more than blur, while SSIM rewards the added structure, which explains the opposite trends. Wind-blown leaves and bushes occluding the lower wall likely contribute.
- **#30 (extrapolation, weakest).** As the last image shows, it has training neighbours on one side only. The largest errors are floaters in the upper-right sky, which have no texture and no SfM points, and the tree at the right edge. The building itself, including the cupola, renders well.



Figure 8: Training views #1 and #15 for comparison.

Training views reach about 27.5 dB with uniformly low error. View #1 shows the same edge effect as #30 at the opposite end of the trajectory: its left-edge porch and trees, seen by only a few cameras, remain blurry.

4. Elapsed time

Table 5: Elapsed time for one full run (30 images, exhaustive matching).

Stage	Time
COLMAP extraction / matching / mapping	149.0 s / 626.7 s / 113.7 s
COLMAP total	14.8 min
GS training (6000 iterations)	22.6 min
GS rendering, 30 views (@2000 / @6000)	49.0 s / 51.6 s

Total (COLMAP + training + rendering @6000)	**38.3 min**

The full run fits within the 40-minute target. COLMAP ran on the CPU, with matching taking 70% of its time. GS training slowed from about 6.4 to 4.1 iterations per second as densification grew the model roughly 4× (checkpoint size: 285 MB -> 1.1 GB).

5. Conclusions and Limitations

- COLMAP reconstructed both subsets completely, with sub-pixel error and consistent self-calibration.
- For this ordered capture, sequential matching gives the same quality as exhaustive at much lower cost and the benefit grows with the number of images.
- More images produced a denser, better-constrained model; the slight rise in reprojection error is not a loss of quality.
- GS improved steadily up to 6000 iterations. Interpolated views generalize well, while the corner and end-of-path views suffer from limited coverage, unconstrained sky and moving bushes.
- **Limitations:** The test images contributed to the COLMAP points used for initialization and to their own poses, so test scores are somewhat optimistic. Three test views make the mean fragile and 6000 iterations is an early stage of training (the default is 30000).

```
!nvidia-smi --query-gpu=name,memory.total --format=csv
import os, time, shutil, sys
import numpy as np, pandas as pd
from google.colab import drive
drive.mount('/content/drive')
```

```
name, memory.total [MiB]
Tesla T4, 15360 MiB
Mounted at /content/drive
```

```
SAVE = "/content/drive/MyDrive/hw3_colmap"
COLMAP_RUN = f"{SAVE}/30_exhaustive_SIMPLE_PINHOLE"
assert os.path.exists(f"{COLMAP_RUN}/sparse/0/images.bin"),"COLMAP model not found on Drive"
```

```
ZIP = f"{SAVE}/south-building-hw3-subsets-15-30.zip"
if not os.path.exists(ZIP):
    from google.colab import files
    up = files.upload()
    shutil.copy(list(up)[0], ZIP)
!unzip -q -o "{ZIP}" -d /content/data
!ls /content/data/images30 | wc -l
```

```
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving south-building-hw3-subsets-15-30.zip to south-building-hw3-subsets-15-30.zip
30
```

```
%cd /content
!git clone -q https://github.com/graphdeco-inria/gaussian-splatting
%cd /content/gaussian-splatting
!git submodule update --init --recursive submodules/diff-gaussian-rasterization submodules/simple-knn
!nvcc --version | tail -1
!python -c "import torch; print('torch', torch.__version__, 'cuda', torch.version.cuda)"

!grep -q cstdint submodules/diff-gaussian-rasterization/cuda_rasterizer/rasterizer_impl.h || \
  sed -i '1i #include <cstdint>' submodules/diff-gaussian-rasterization/cuda_rasterizer/rasterizer_impl.h
!grep -q cfloat submodules/simple-knn/simple_knn.cu || sed -i '1i #include <cfloat>' submodules/simple-knn/simple_knn.cu

t0 = time.perf_counter()
!TORCH_CUDA_ARCH_LIST="7.5" pip install -q --no-build-isolation ./submodules/diff-gaussian-rasterization ./submodules/simple-knn
!pip install -q plyfile
print(f"Build time: {(time.perf_counter()-t0)/60:.1f} min")
!python -c "import diff_gaussian_rasterization, simple_knn._C; print('CUDA extensions OK')"
```

```
/content
/content/gaussian-splatting
Submodule 'submodules/diff-gaussian-rasterization' (https://github.com/graphdeco-inria/diff-gaussian-rasterization.git) registered for path 'submodules/di
Submodule 'submodules/simple-knn' (https://gitlab.inria.fr/bkerbl/simple-knn.git) registered for path 'submodules/simple-knn'
Cloning into '/content/gaussian-splatting/submodules/diff-gaussian-rasterization'...
Cloning into '/content/gaussian-splatting/submodules/simple-knn'...
Submodule path 'submodules/diff-gaussian-rasterization': checked out '9c5c2028f6fbee2be239bc4c9421ff894fe4fbe0'
```

```
Submodule 'third_party/glm' (https://github.com/g-truc/glm.git) registered for path 'submodules/diff-gaussian-rasterization/third_party/glm'
Cloning into '/content/gaussian-splatting/submodules/diff-gaussian-rasterization/third_party/glm'...
Submodule path 'submodules/diff-gaussian-rasterization/third_party/glm': checked out '5c46b9c07008ae65cb81ab79cd677ecc1934b903'
Submodule path 'submodules/simple-knn': checked out '86710c2d4b46680c02301765dd79e465819c8f19'
Build cuda_13.0.r13.0/compiler.36424714_0
torch 2.11.0+cu130 cuda 13.0
  Preparing metadata (setup.py) ... done
  Preparing metadata (setup.py) ... done
  Building wheel for diff_gaussian_rasterization (setup.py) ... done
  Building wheel for simple_knn (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.3/43.3 kB 2.9 MB/s eta 0:00:00
Build time: 4.3 min
CUDA extensions OK
```

```
GS = "/content/gaussian-splatting"
DST = "/content/gs_data/south30"
OUT = "/content/gs_out/south30"
TEST = ["P1180198.JPG", "P1180208.JPG", "P1180218.JPG"]

shutil.rmtree(DST, ignore_errors=True)
shutil.copytree("/content/data/images30", f"{DST}/images")
shutil.copytree(f"{COLMAP_RUN}/sparse/0", f"{DST}/sparse/0")
with open(f"{DST}/sparse/0/test.txt", "w") as f:
    f.write("\n".join(TEST) + "\n")

all_names = sorted(os.listdir(f"{DST}/images"))
print("Held-out numbers:", [all_names.index(n) + 1 for n in TEST])

!sed -i 's/train_test_exp, llffhold=8):/train_test_exp, llffhold=0):/' {GS}/scene/dataset_readers.py
!grep -n "def readColmapSceneInfo" {GS}/scene/dataset_readers.py

sys.path.insert(0, GS)
from scene.dataset_readers import readColmapSceneInfo
si =readColmapSceneInfo(DST, "images", "",True,False)
train_names = sorted(c.image_name for c in si.train_cameras)
test_names= sorted(c.image_name for c in si.test_cameras)
print(len(train_names), "train |", len(test_names), "test:",test_names)
```

```
Held-out numbers: [10, 20, 30]
145:def readColmapSceneInfo(path, images, depths, eval, train_test_exp, llffhold=0):
Reading camera 30/30
Converting point3d.bin to .ply, will happen only the first time you open the scene.
27 train | 3 test: ['P1180198.JPG', 'P1180208.JPG', 'P1180218.JPG']
```

```
t0 = time.perf_counter()
!cd {GS} && python train.py -s {DST} -m {OUT} --eval --iterations 6000 \
    --save_iterations 2000 6000 --checkpoint_iterations 2000 6000 \
    --test_iterations 2000 6000 --disable_viewer 2>&1 | tee /content/gs_train.log
t_train = time.perf_counter() - t0
print(f"GS training wall time: {t_train/60:.1f} min")
!grep "Evaluating\|Saving" /content/gs_train.log
!ls -lh {OUT} {OUT}/point_cloud
```

```
2026-10-05 03:52:36.114303: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Optimizing /content/gs_out/south30
```

```
Output folder: /content/gs_out/south30 [05/10 03:52:40]
Reading camera 30/30 [05/10 03:52:41]
Loading Training Cameras [05/10 03:52:41]
Loading Test Cameras [05/10 03:52:44]
Number of points at initialisation :  33207 [05/10 03:52:44]
Training progress: 100%|██████████| 6000/6000 [21:31<00:00,  4.65it/s, Loss=0.0560911, Depth Loss=0.0000000]

[ITER 2000] Evaluating test: L1 0.07431963955362637 PSNR 19.94571749369303 [05/10 03:57:58]

[ITER 2000] Evaluating train: L1 0.03602591268718243 PSNR 24.3357666015625 [05/10 03:58:10]

[ITER 2000] Saving Gaussians [05/10 03:58:11]

[ITER 2000] Saving Checkpoint [05/10 03:58:15]

[ITER 6000] Evaluating test: L1 0.050627533346414566 PSNR 22.28899828592936 [05/10 04:14:25]

[ITER 6000] Evaluating train: L1 0.0240146990865469 PSNR 27.74561920166016 [05/10 04:14:31]

[ITER 6000] Saving Gaussians [05/10 04:14:31]

[ITER 6000] Saving Checkpoint [05/10 04:14:56]

Training complete. [05/10 04:15:07]
GS training wall time: 22.6 min
[ITER 2000] Evaluating test: L1 0.07431963955362637 PSNR 19.94571749369303 [05/10 03:57:58]
[ITER 2000] Evaluating train: L1 0.03602591268718243 PSNR 24.3357666015625 [05/10 03:58:10]
[ITER 2000] Saving Gaussians [05/10 03:58:11]
[ITER 2000] Saving Checkpoint [05/10 03:58:15]
[ITER 6000] Evaluating test: L1 0.050627533346414566 PSNR 22.28899828592936 [05/10 04:14:25]
[ITER 6000] Evaluating train: L1 0.0240146990865469 PSNR 27.74561920166016 [05/10 04:14:31]
[ITER 6000] Saving Gaussians [05/10 04:14:31]
[ITER 6000] Saving Checkpoint [05/10 04:14:56]
/content/gs_out/south30:
total 1.5G
-rw-r--r-- 1 root root  12K Oct  5 03:52 cameras.json
-rw-r--r-- 1 root root  220 Oct  5 03:52 cfg_args
-rw-r--r-- 1 root root 285M Oct  5 03:58 chkpnt2000.pth
-rw-r--r-- 1 root root 1.1G Oct  5 04:15 chkpnt6000.pth
-rw-r--r-- 1 root root  57M Oct  5 04:14 events.out.tfevents.1791172360.77bf501b6e3b.3942.0
-rw-r--r-- 1 root root 5.1K Oct  5 04:14 exposure.json
-rw-r--r-- 1 root root 876K Oct  5 03:52 input.ply
drwxr-xr-x 4 root root 4.0K Oct  5 04:14 point_cloud

/content/gs_out/south30/point_cloud:
total 8.0K
drwxr-xr-x 2 root root 4.0K Oct  5 03:58 iteration_2000
drwxr-xr-x 2 root root 4.0K Oct  5 04:14 iteration_6000
```

```
t_render = {}
for it in [2000, 6000]:
    t0 = time.perf_counter()
    !cd {GS} && python render.py -m {OUT} --iteration {it}
    t_render[it] = time.perf_counter()-t0
    print(f"Render @ {it}: {t_render[it]:.1f}s for {len(train_names)+len(test_names)} views")
```

```
Looking for config file in /content/gs_out/south30/cfg_args
Config file found: /content/gs_out/south30/cfg_args
Rendering /content/gs_out/south30
```

```
Loading trained model at iteration 2000 [05/10 04:15:27]
Reading camera 30/30 [05/10 04:15:28]
Loading Training Cameras [05/10 04:15:28]
Loading Test Cameras [05/10 04:15:30]
Rendering progress: 100% 27/27 [00:35<00:00,  1.30s/it]
Rendering progress: 100% 3/3 [00:04<00:00,  1.47s/it]
Render @ 2000: 49.0s for 30 views
Looking for config file in /content/gs_out/south30/cfg_args
Config file found: /content/gs_out/south30/cfg_args
Rendering /content/gs_out/south30
Loading trained model at iteration 6000 [05/10 04:16:15]
Reading camera 30/30 [05/10 04:16:15]
Loading Training Cameras [05/10 04:16:15]
Loading Test Cameras [05/10 04:16:17]
Rendering progress: 100% 27/27 [00:38<00:00,  1.43s/it]
Rendering progress: 100% 3/3 [00:03<00:00,  1.32s/it]
Render @ 6000: 51.6s for 30 views
```

```
!cd {GS} && python metrics.py -m {OUT}

import torch
from PIL import Image
import torchvision.transforms.functional as tf
from utils.loss_utils import ssim
from utils.image_utils import psnr

def to_t(p): return tf.to_tensor(Image.open(p))[:3].unsqueeze(0).cuda()

rows = []
with torch.no_grad():
    for split, names in [("train",train_names), ("test",test_names)]:
        for it in [2000, 6000]:
            d = f"{OUT}/{split}/ours_{it}"
            fns = sorted(os.listdir(f"{d}/renders"))
            assert len(fns) == len(names)
            for fn,name in zip(fns, names):
                r,g = to_t(f"{d}/renders/{fn}"), to_t(f"{d}/gt/{fn}")
                rows.append(dict(split=split, iteration=it, image=name,
                                 number=all_names.index(name)+1,
                                 PSNR=psnr(r,g).mean().item(), SSIM=ssim(r,g).item()))
df = pd.DataFrame(rows)
df.to_csv(f"{SAVE}/gs_metrics_per_view.csv", index=False)

summary = df.groupby(["split","iteration"])[["PSNR","SSIM"]].mean().round(4)
print(summary)
print(df[df.split == "test"].pivot(index="number",columns="iteration", values=["PSNR", "SSIM"]).round(3))
```

```
  0% 0.00/528M [00:00<?, ?B/s]
  2% 11.9M/528M [00:00<00:04, 124MB/s]
  5% 28.8M/528M [00:00<00:03, 154MB/s]
  9% 49.1M/528M [00:00<00:02, 181MB/s]
 13% 69.0M/528M [00:00<00:02, 191MB/s]
 17% 87.4M/528M [00:00<00:02, 184MB/s]
```

```
 33% 175M/528M [00:01<00:02, 174MB/s]
 37% 194M/528M [00:01<00:01, 179MB/s]
 40% 213M/528M [00:01<00:01, 185MB/s]
 44% 232M/528M [00:01<00:01, 188MB/s]
 47% 250M/528M [00:01<00:01, 179MB/s]
 51% 269M/528M [00:01<00:01, 185MB/s]
 54% 287M/528M [00:01<00:01, 187MB/s]
 58% 305M/528M [00:01<00:01, 187MB/s]
 61% 323M/528M [00:01<00:01, 187MB/s]
 65% 341M/528M [00:01<00:01, 180MB/s]
 68% 358M/528M [00:04<00:08, 22.1MB/s]
 71% 374M/528M [00:04<00:05, 29.0MB/s]
 75% 393M/528M [00:04<00:03, 40.1MB/s]
 79% 416M/528M [00:04<00:02, 56.2MB/s]
 83% 438M/528M [00:04<00:01, 75.1MB/s]
 86% 456M/528M [00:05<00:00, 84.6MB/s]
 90% 473M/528M [00:05<00:00, 97.5MB/s]
 93% 490M/528M [00:05<00:00, 112MB/s]
 96% 507M/528M [00:05<00:00, 124MB/s]
100% 528M/528M [00:05<00:00, 99.5MB/s]
Downloading: "https://raw.githubusercontent.com/richzhang/PerceptualSimilarity/master/lpips/weights/v0.1/vgg.pth" to /root/.cache/torch/hub/checkpoints/

100% 7.12k/7.12k [00:00<00:00, 20.2MB/s]
Metric evaluation progress: 100% 3/3 [00:14<00:00,  4.67s/it]
  SSIM :    0.7659714
  PSNR :   22.2865982
  LPIPS:    0.2212496

Method: ours_2000
Metric evaluation progress: 100% 3/3 [00:05<00:00,  1.98s/it]
  SSIM :    0.6541004
  PSNR :   19.9444962
  LPIPS:    0.3673497

                    PSNR    SSIM
split iteration
test  2000       19.9445  0.6541
      6000       22.2866  0.7660
train 2000       23.8774  0.7015
      6000       27.2893  0.8405
             PSNR           SSIM
iteration    2000    6000   2000   6000
number
10         23.597  22.558  0.688  0.759
20         24.573  26.811  0.722  0.840
30         11.664  17.491  0.552  0.699
```

```
import matplotlib.pyplot as plt

def compare(split, names, picks, fname):
    fig, axes = plt.subplots(len(picks), 4, figsize=(20, 3.9 * len(picks)))
    axes = np.atleast_2d(axes)
    for row, name in zip(axes, picks):
        fn = f"{names.index(name):05d}.png"
        ld = lambda it, kind: np.asarray(Image.open(f"{OUT}/{split}/ours_{it}/{kind}/{fn}"))[..., :3] / 255.
        gt, r2, r6 = ld(6000, "gt"), ld(2000, "renders"), ld(6000, "renders")
        m = df[(df.image == name) & (df.split == split)].set_index("iteration")
        n = all_names.index(name) + 1
        for ax, im, t in zip(row[:3], [gt, r2, r6],
                             [f"#{n} ground truth ({split})",
```

```
                            f"iter 2000: PSNR {m.PSNR[2000]:.2f}, SSIM {m.SSIM[2000]:.3f}",
                            f"iter 6000: PSNR {m.PSNR[6000]:.2f}, SSIM {m.SSIM[6000]:.3f}"]):
            ax.imshow(im); ax.set_title(t); ax.axis("off")
        row[3].imshow(np.abs(r6 - gt).mean(-1), cmap="inferno", vmin=0, vmax=0.3)
        row[3].set_title("|render − GT| @ 6000")
        row[3].axis("off")
    plt.tight_layout(); plt.savefig(f"{SAVE}/{fname}", dpi=150); plt.show()

compare("test",test_names,test_names,"gs_test_views.png")
compare("train",train_names,[train_names[0],train_names[13]],"gs_train_views.png")
```

```
t_colmap = 889.3
print(f"COLMAP:          {t_colmap/60:.1f} min")
print(f"GS training:     {t_train/60:.1f} min")
for it, t in t_render.items(): print(f"GS render @{it}: {t:.1f} s")
print(f"COLMAP + GS train + render@6000: {(t_colmap + t_train + t_render[6000])/60:.1f} min")
```

```
!nvidia-smi
```

```
Mon Oct  5 00:53:49 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   35C    P8             11W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
!apt-get update -qq
!apt-get install -y -qq colmap
!colmap -h | head -20
```

```
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os

folder = "/content/drive/MyDrive/south-building-hw3-subsets-15-30"
os.listdir(folder)
```

```
['README.txt', 'images15', 'images30', 'selection.json']
```

```
from google.colab import files
up = files.upload()
!mkdir -p /content/data && unzip -q -o south-building-hw3-subsets-15-30.zip -d /content/data
!ls /content/data && ls /content/data/images15 | wc -l && ls /content/data/images30 | wc -l
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving south-building-hw3-subsets-15-30.zip to south-building-hw3-subsets-15-30.zip
images15  images30  README.txt   selection.json
15
30
```

```
import subprocess, time, os, shutil, re

def sh(cmd):
    t = time.perf_counter()
    r = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    dt = time.perf_counter() - t
    if r.returncode != 0:
```

```
        print(r.stdout[-1500:], r.stderr[-1500:])
        raise RuntimeError(f"Failed: {cmd}")
    return dt, r.stdout + r.stderr

def run_colmap(n, matcher, cam="SIMPLE_PINHOLE", extra_match=""):
    images = f"/content/data/images{n}"
    ws = f"/content/runs/{n}_{matcher}_{cam}"
    shutil.rmtree(ws, ignore_errors=True)
    os.makedirs(f"{ws}/sparse")
    db = f"{ws}/database.db"
    times = {}

    times["extract"], _ = sh(
        f"colmap feature_extractor --database_path {db} --image_path {images} "
        f"--ImageReader.camera_model {cam} --ImageReader.single_camera 1 "
        f"--SiftExtraction.use_gpu 0")
    times["match"], _ = sh(
        f"colmap {matcher}_matcher --database_path {db} "
        f"--SiftMatching.use_gpu 0 {extra_match}")
    times["map"], _ = sh(
        f"colmap mapper --database_path {db} --image_path {images} "
        f"--output_path {ws}/sparse")

    models = sorted(os.listdir(f"{ws}/sparse"))
    _, stats = sh(f"colmap model_analyzer --path {ws}/sparse/0")
    times["total"] = sum(times.values())

    print(f"=== {n} images | {matcher} | {cam} ===")
    print("Sub-models produced:", models)
    print("\n".join(l for l in stats.splitlines()
                    if re.search(r"Cameras|Images|Registered|Points|Observations|Mean", l)))
    print({k: f"{v:.1f}s" for k, v in times.items()})
    return ws, times

ws, t = run_colmap(15, "exhaustive")
```

```
=== 15 images | exhaustive | SIMPLE_PINHOLE ===
Sub-models produced: ['0']
I20261005 01:20:23.801056 22903 model.cc:429] Cameras: 1
I20261005 01:20:23.801132 22903 model.cc:430] Images: 15
I20261005 01:20:23.801141 22903 model.cc:431] Registered images: 15
I20261005 01:20:23.801147 22903 model.cc:433] Points: 13350
I20261005 01:20:23.801154 22903 model.cc:434] Observations: 50742
I20261005 01:20:23.801162 22903 model.cc:436] Mean track length: 3.800899
I20261005 01:20:23.801172 22903 model.cc:438] Mean observations per image: 3382.800000
I20261005 01:20:23.801180 22903 model.cc:441] Mean reprojection error: 0.300908px
{'extract': '76.5s', 'match': '160.6s', 'map': '23.9s', 'total': '261.1s'}
```

```
import pandas as pd, json
from google.colab import drive
drive.mount('/content/drive')
SAVE = "/content/drive/MyDrive/hw3_colmap"
os.makedirs(SAVE, exist_ok=True)

def parse_stats(ws):
    _, s = sh(f"colmap model_analyzer --path {ws}/sparse/0")
    grab = lambda key: float(re.search(rf"{key}: ([\d.]+)", s).group(1))
```

```
    grab = lambda key: float(re.search(rf"{key}: ([\d.]+)", s).group(1))
    return {"registered": int(grab("Registered images")),
            "points": int(grab("Points")),
            "track_len": round(grab("Mean track length"), 2),
            "reproj_px": round(grab("Mean reprojection error"), 3),
            "n_models": len(os.listdir(f"{ws}/sparse"))}

results = []
def record(n, matcher, cam="SIMPLE_PINHOLE", extra="", label=None):
    ws, t = run_colmap(n, matcher, cam, extra)
    row = {"config":label or f"{n}-{matcher}", "images":n, **parse_stats(ws),
           **{f"t_{k}": round(v,1) for k,v in t.items()}}
    results.append(row)
    pd.DataFrame(results).to_csv(f"{SAVE}/colmap_results.csv", index=False)
    shutil.copytree(ws, f"{SAVE}/{os.path.basename(ws)}", dirs_exist_ok=True)
    return ws

record(15, "exhaustive")
record(15, "sequential")
record(30, "sequential")
record(30, "exhaustive")
pd.DataFrame(results)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
=== 15 images | exhaustive | SIMPLE_PINHOLE ===
Sub-models produced: ['0']
I20261005 01:26:23.339318 24460 model.cc:429] Cameras: 1
I20261005 01:26:23.339413 24460 model.cc:430] Images: 15
I20261005 01:26:23.339422 24460 model.cc:431] Registered images: 15
I20261005 01:26:23.339430 24460 model.cc:433] Points: 13364
I20261005 01:26:23.339437 24460 model.cc:434] Observations: 50764
I20261005 01:26:23.339444 24460 model.cc:436] Mean track length: 3.798563
I20261005 01:26:23.339454 24460 model.cc:438] Mean observations per image: 3384.266667
I20261005 01:26:23.339463 24460 model.cc:441] Mean reprojection error: 0.303533px
{'extract': '76.0s', 'match': '157.5s', 'map': '24.2s', 'total': '257.7s'}
=== 15 images | sequential | SIMPLE_PINHOLE ===
Sub-models produced: ['0']
I20261005 01:30:18.820041 25494 model.cc:429] Cameras: 1
I20261005 01:30:18.820142 25494 model.cc:430] Images: 15
I20261005 01:30:18.820152 25494 model.cc:431] Registered images: 15
I20261005 01:30:18.820161 25494 model.cc:433] Points: 13314
I20261005 01:30:18.820169 25494 model.cc:434] Observations: 50629
I20261005 01:30:18.820178 25494 model.cc:436] Mean track length: 3.802689
I20261005 01:30:18.820189 25494 model.cc:438] Mean observations per image: 3375.266667
I20261005 01:30:18.820200 25494 model.cc:441] Mean reprojection error: 0.302494px
{'extract': '76.9s', 'match': '135.6s', 'map': '22.7s', 'total': '235.1s'}
=== 30 images | sequential | SIMPLE_PINHOLE ===
Sub-models produced: ['0']
I20261005 01:40:08.938524 28060 model.cc:429] Cameras: 1
I20261005 01:40:08.938704 28060 model.cc:430] Images: 30
I20261005 01:40:08.938728 28060 model.cc:431] Registered images: 30
I20261005 01:40:08.938748 28060 model.cc:433] Points: 32755
I20261005 01:40:08.938766 28060 model.cc:434] Observations: 171622
I20261005 01:40:08.938789 28060 model.cc:436] Mean track length: 5.239566
I20261005 01:40:08.938810 28060 model.cc:438] Mean observations per image: 5720.733333
I20261005 01:40:08.938836 28060 model.cc:441] Mean reprojection error: 0.331357px
{'extract': '147.3s', 'match': '341.6s', 'map': '100.7s', 'total': '589.6s'}
=== 30 images | exhaustive | SIMPLE_PINHOLE ===
Sub-models produced: ['0']
I20261005 01:54:58.887336 31976 model.cc:429] Cameras: 1
I20261005 01:54:58.887427 31976 model.cc:430] Images: 30
I20261005 01:54:58.887435 31976 model.cc:431] Registered images: 30
I20261005 01:54:58.887444 31976 model.cc:433] Points: 33207
I20261005 01:54:58.887449 31976 model.cc:434] Observations: 172720
I20261005 01:54:58.887458 31976 model.cc:436] Mean track length: 5.201313
I20261005 01:54:58.887468 31976 model.cc:438] Mean observations per image: 5757.333333
I20261005 01:54:58.887481 31976 model.cc:441] Mean reprojection error: 0.335429px
{'extract': '149.0s', 'match': '626.7s', 'map': '113.7s', 'total': '889.3s'}
```

	config	images	registered	points	track_len	reproj_px	n_models	t_extract	t_match	t_map	t_total
0	15-exhaustive	15	15	13364	3.80	0.304	1	76.0	157.5	24.2	257.7
1	15-sequential	15	15	13314	3.80	0.302	1	76.9	135.6	22.7	235.1
2	30-sequential	30	30	32755	5.24	0.331	1	147.3	341.6	100.7	589.6
3	30-exhaustive	30	30	33207	5.20	0.335	1	149.0	626.7	113.7	889.3

```
import sqlite3, numpy as np, cv2, matplotlib.pyplot as plt
MAXID = 2147483647
```

```
def load_db(db):
    con = sqlite3.connect(db)
    names =dict(con.execute("SELECT image_id, name FROM images"))
    kps = {i: np.frombuffer(d, np.float32).reshape(r, c)[:, :2]
           for i, r, c, d in con.execute("SELECT image_id, rows, cols, data FROM keypoints") if d}
    def pairs(table):
        out = {}
        for pid, r, c, d in con.execute(f"SELECT pair_id, rows, cols, data FROM {table}"):
            if d and r:
                i2 = pid % MAXID
                out[((pid - i2)//MAXID, i2)] = np.frombuffer(d, np.uint32).reshape(r, c)
        return out
    raw, verified = pairs("matches"), pairs("two_view_geometries")
    con.close()
    order =sorted(names, key=lambda i: names[i])
    return names, order, kps, raw, verified

def get_pair(D, ia, ib):
    M= D.get((min(ia, ib), max(ia, ib)), np.zeros((0, 2), np.uint32))
    return M if ia < ib else M[:, ::-1]

def img(n, name):
    return cv2.cvtColor(cv2.imread(f"/content/data/images{n}/{name}"),cv2.COLOR_BGR2RGB)
```

```
WS = "/content/runs/30_exhaustive_SIMPLE_PINHOLE"
names, order, kps, raw, ver = load_db(f"{WS}/database.db")

k = 1
iid = order[k-1]
plt.figure(figsize=(10, 7.5))
plt.imshow(img(30,names[iid]))
plt.scatter(*kps[iid].T,s=1.5,c="lime")
plt.title(f"Image #{k}: {len(kps[iid])} SIFT keypoints"); plt.axis("off")
print("Mean keypoints/image:", int(np.mean([len(v) for v in kps.values()])))
```

```
Mean keypoints/image: 14173
```



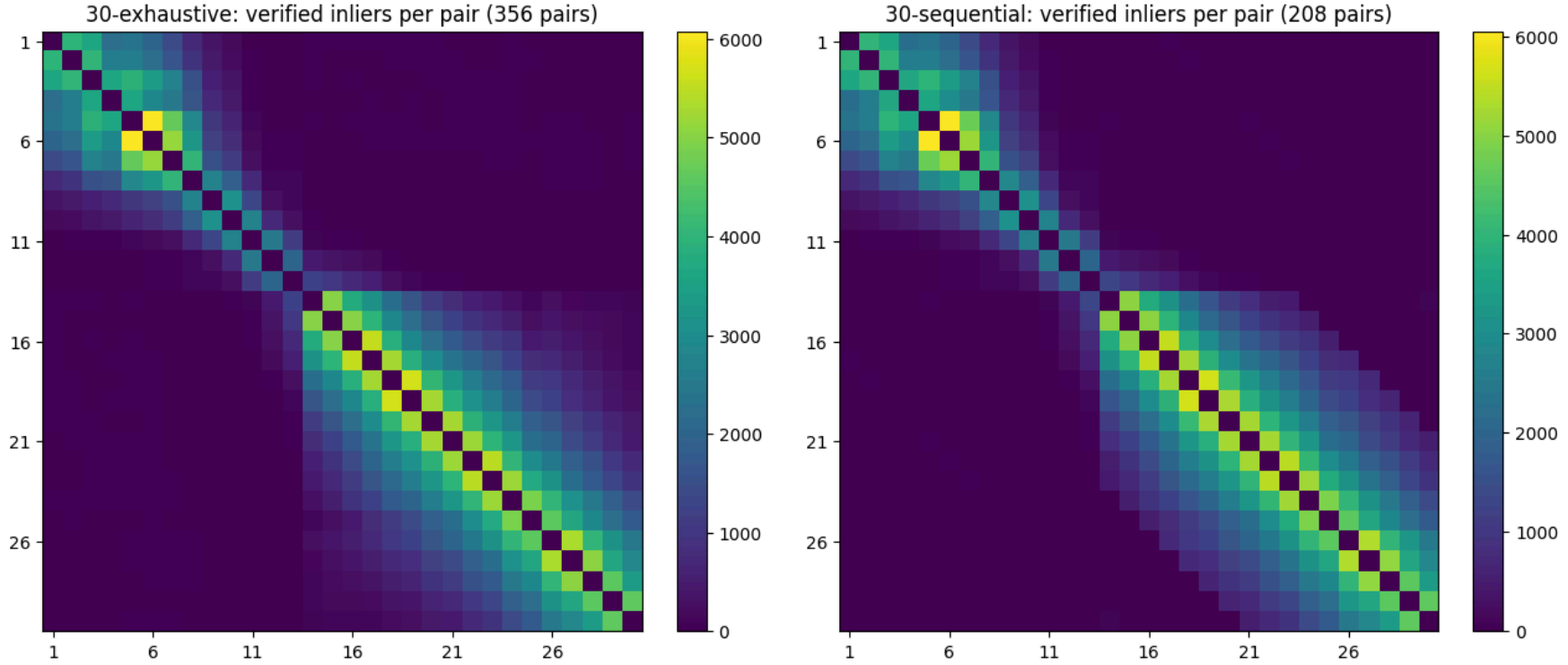

```
def show_matches(a, b, max_lines=200, seed=0):
    ia, ib = order[a-1], order[b-1]
    R, V= get_pair(raw, ia, ib), get_pair(ver, ia, ib)
    A, B = img(30, names[ia]), img(30, names[ib])
    canvas = np.hstack([A, B]); w = A.shape[1]
    sel =np.random.default_rng(seed).permutation(len(V))[:max_lines]
    plt.figure(figsize=(16,6)); plt.imshow(canvas)
    for i, j in V[sel]:
        (x1,y1), (x2,y2) = kps[ia][i], kps[ib][j]
        plt.plot([x1,x2 + w], [y1,y2], lw=0.6)
    plt.title(f"#{a} ↔ #{b}: {len(R)} raw matches, {len(V)} verified inliers "
              f"(showing {min(max_lines, len(V))})"); plt.axis("off")

show_matches(1,2)
show_matches(1,12)
```




```
fig,axes = plt.subplots(1, 2, figsize=(13,5.5))
```

```
for ax, mode in zip(axes, ['exhaustive', 'sequential']):
    nm, od, _, _, vr = load_db(f"/content/runs/30_{mode}_SIMPLE_PINHOLE/database.db")
    N = len(od); pos = {iid: p for p,iid in enumerate(od)}
    Mtx = np.zeros((N, N))
    for (i1, i2), m in vr.items():
        Mtx[pos[i1], pos[i2]] = Mtx[pos[i2], pos[i1]] = len(m)
    im = ax.imshow(Mtx, cmap="viridis")
    ax.set_title(f"30-{mode}: verified inliers per pair ({len(vr)} pairs)")
    ax.set_xticks(range(0,N,5), range(1,N+1,5)); ax.set_yticks(range(0,N,5), range(1,N+1,5))
    fig.colorbar(im,  ax=ax, fraction=0.046)
plt.tight_layout()
```




```
from scipy.spatial.transform import Rotation as Rot
import plotly.graph_objects as go

def read_model(ws):
    txt = f"{ws}/sparse_txt"; os.makedirs(txt, exist_ok=True)
    sh(f"colmap model_converter --input_path {ws}/sparse/0 --output_path {txt} --output_type TXT")
    cam = [l.split() for l in open(f"{txt}/cameras.txt") if not l.startswith("#") and l.strip()][0]
    P = np.array([l.split()[:8] for l in open(f"{txt}/points3D.txt") if not l.startswith("#")], float)
    lines = [l for l in open(f"{txt}/images.txt") if not l.startswith("#")]
    C, D, U, N = [], [], [], []
    for l in lines[0::2]:
        e = l.split()
        qw, qx, qy, qz = map(float, e[1:5]); t = np.array(e[5:8], float)
        R = Rot.from_quat([qx, qy, qz, qw]).as_matrix()
```

```
        C.append(-R.T @ t)
        D.append(R.T @ [0, 0, 1])
        U.append(-R.T @ [0, 1, 0])
        N.append(e[9])
    o = np.argsort(N)
    return cam,P,np.array(C)[o],np.array(D)[o],np.array(U)[o]

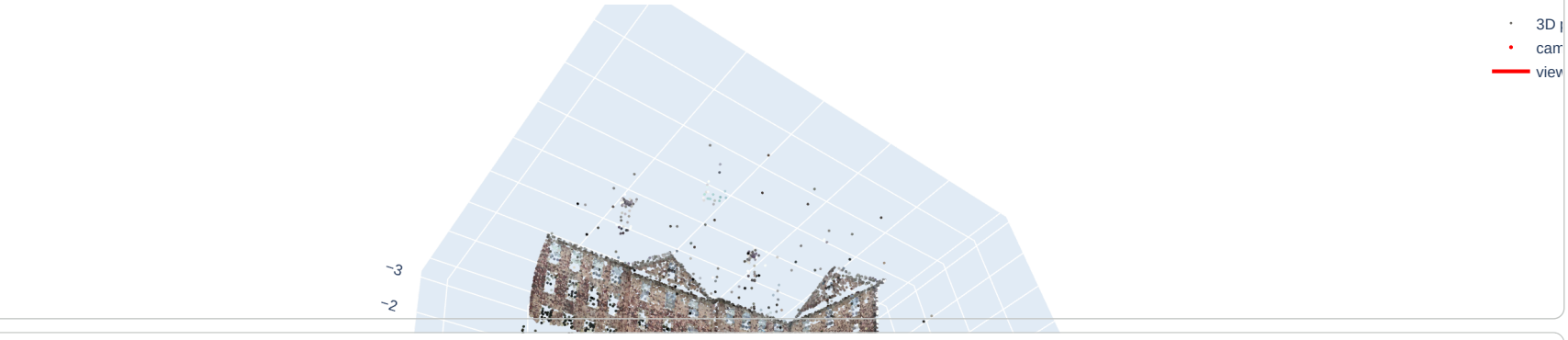
def clean(P, keep=98):
    d = np.linalg.norm(P[:, 1:4] - np.median(P[:, 1:4], 0), axis=1)
    return P[d<np.percentile(d, keep)]

for mode in ["15_exhaustive", "15_sequential", "30_sequential", "30_exhaustive"]:
    cam, *_ = read_model(f"/content/runs/{mode}_SIMPLE_PINHOLE")
    print(f"{mode:15s} f = {float(cam[4]):7.1f}px  cx = {float(cam[5]):.1f}  cy = {float(cam[6]):.1f}")
```

```
15_exhaustive   f =  1341.5px  cx = 800.0  cy = 600.0
15_sequential   f =  1342.2px  cx = 800.0  cy = 600.0
30_sequential   f =  1345.1px  cx = 800.0  cy = 600.0
30_exhaustive   f =  1345.1px  cx = 800.0  cy = 600.0
```

```
def plot3d(ws, title):
    cam,P,C,D,U = read_model(ws); P = clean(P)
    s = 0.06 * np.ptp(P[:, 1:4], 0).max()
    up = U.mean(0) / np.linalg.norm(U.mean(0))
    fig = go.Figure()
    fig.add_scatter3d(x=P[:,1], y=P[:,2], z=P[:,3], mode="markers", name="3D points",
        marker=dict(size=1.2, color=[f"rgb({r:.0f},{g:.0f},{b:.0f})" for r, g, b in P[:, 4:7]]))
    fig.add_scatter3d(x=C[:,0], y=C[:,1], z=C[:,2], mode="markers+text", name="cameras",
        text=[str(i + 1) for i in range(len(C))], marker=dict(size=3, color="red"))
    L = np.full((3 * len(C),3), np.nan); L[0::3] = C; L[1::3] = C + s * D
    fig.add_scatter3d(x=L[:,0], y=L[:,1], z=L[:,2], mode="lines", name="view dir",
        line=dict(color="red", width=3))
    fig.update_layout(title=f"{title} ({len(P)} pts shown, f={float(cam[4]):.0f}px)", height=700,
        scene=dict(aspectmode="data", camera=dict(up=dict(x=up[0], y=up[1], z=up[2]))))
    fig.show()

plot3d("/content/runs/30_sequential_SIMPLE_PINHOLE", "30-sequential sparse model")
```




```
def topdown(ax, ws, title):
    cam, P, C, D, U = read_model(ws)
    P = clean(P)
    u = U.mean(0); u/= np.linalg.norm(u)
    c0 = C.mean(0)
    Cp = (C c0)-np.outer((C-c0) @ u, u)
    e1 = np.linalg.svd(Cp, full_matrices=False)[2][0]; e2 = np.cross(u, e1)
    to2d = lambda X: np.stack([(X-c0) @ e1, (X-c0) @ e2], 1)
    p2, c2 = to2d(P[:, 1:4]), to2d(C)
    d2 = np.stack([D @ e1, D @ e2], 1)
    d2 /= np.linalg.norm(d2, axis=1, keepdims=True)
    Lq = 0.08 * np.ptp(p2, 0).max()
    ax.scatter(*p2.T, s=0.2, c=P[:, 4:7] / 255)
    ax.plot(*c2.T, "r-", lw=0.8)
    ax.quiver(*c2.T, *(d2 * Lq).T, color="red", angles="xy", scale_units="xy", scale=1, width=0.004)
    for i, (x,y) in enumerate(c2): ax.annotate(str(i+1), (x,y), fontsize=7)
    ax.set_aspect("equal"); ax.set_title(title); ax.axis("off")

fig, axes = plt.subplots(1,2, figsize=(15, 7))
topdown(axes[0], "/content/runs/15_exhaustive_SIMPLE_PINHOLE", "15 images (exhaustive)")
```